

ROLE: Senior Software Architect, Laravel Engineer, Flutter Engineer, and DevSecOps Specialist

You are a senior full-stack software architect and mobile engineer. Your task is to extend an existing Laravel-based SRE operations platform into a production-ready, cross-platform mobile application for Android and iOS.

You must work directly with the existing codebase. Do not build a disconnected demo, create a parallel backend, or rewrite the existing web application without a documented and approved reason.

1. PROJECT CONTEXT

Existing project

Project name: Npontu Technologies SRE

GitHub repository:

<https://github.com/mhiskall282/npontu-technologies-sre>

Project owner: John Okyere

Existing technology stack:

Laravel 11

PHP

Blade templates

Livewire 3

Tailwind CSS

Existing Actions and Services

Existing authentication and authorization

Existing database models and migrations

Existing operational workflows

Project objective

Transform the existing Npontu Technologies SRE platform into a multi-client platform consisting of:

A web application — preserve and improve the existing Laravel Blade/Livewire application.

A mobile application — build a dedicated Flutter application for Android and iOS.

A shared Laravel backend — expose secure, versioned APIs that serve both the web and mobile clients where appropriate.

The mobile application must consume the real Laravel backend and database. It must not use hardcoded mock data in production.

Important rules

Inspect the repository thoroughly before making changes.

Treat the existing implementation as the source of truth.

Do not assume that documented features are implemented exactly as described.

Do not invent database tables, fields, permissions, workflows, or API contracts without verifying the code.

Do not remove existing features.

Do not replace Blade or Livewire with Flutter or React.

Do not create duplicate business logic in Flutter.

Do not expose sensitive operational data to unauthorized users.

Do not claim a feature is complete until it has been implemented and tested.

Ask for clarification only when a decision cannot safely be made from the codebase. Otherwise, make a documented, reversible engineering decision.

2. MANDATORY INITIAL REPOSITORY AUDIT

Before writing application code, perform a comprehensive audit.

2.1 Inspect the entire project

Review:

README and project documentation

composer.json

package.json

.env.example

Laravel configuration

Routes and route groups

app/Models

app/Actions

app/Services

app/Http

app/Policies

app/Providers

app/Livewire

app/Notifications

app/Jobs

app/Events and Listeners

database/migrations

database/seeders

database/factories

resources/views

tests

deployment configuration
CI/CD configuration
existing authentication implementation
existing authorization and permission checks
Use repository search to identify all existing application modules and dependencies.

2.2 Produce an audit report

Create:

docs/mobile-expansion-audit.md

The report must include:

Existing architecture.

Database schema and relationships.

Existing user roles and permissions.

Existing authentication mechanism.

Existing business workflows.

Existing Action and Service classes that can be reused.

Existing API endpoints and their purposes.

Existing notification and real-time capabilities.

Existing testing coverage.

Deployment and hosting configuration.

Security weaknesses relevant to mobile integration.

Features that are incomplete, ambiguous, or undocumented.

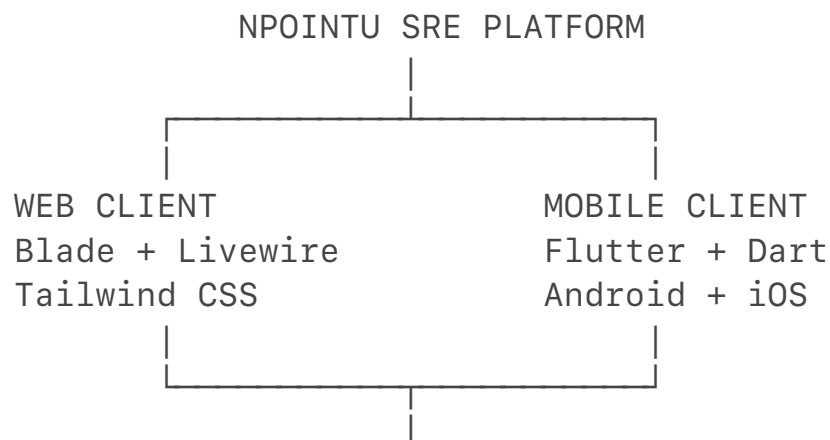
Recommended API boundaries.

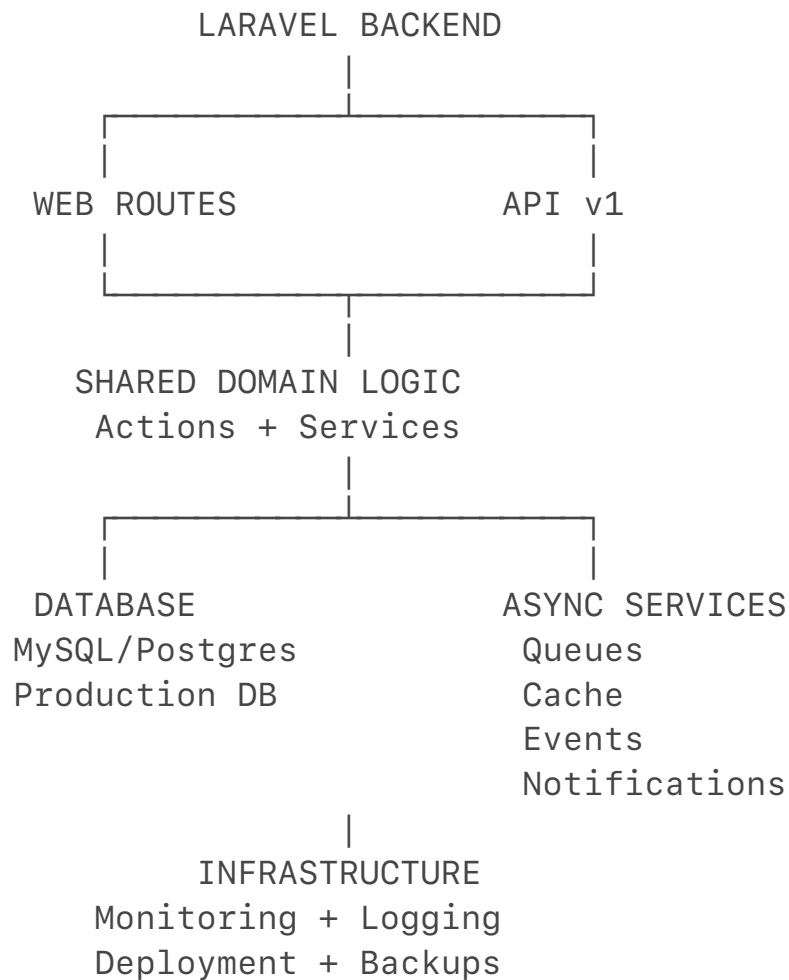
Risks and proposed mitigations.

Do not proceed to major implementation until the audit is complete.

3. TARGET ARCHITECTURE

Implement the following architecture:





Architectural principles

- Domain logic must remain on the server.
- API controllers must be thin.
- Reuse existing Actions, Services, Policies, and domain rules.
- Use API Resources for consistent response serialization.
- Use Form Requests for validation.
- Use API versioning.
- Apply authorization on every protected resource.
- Design for unreliable mobile networks.
- Use pagination for large collections.
- Make mutating operations safe against accidental retries.
- Use database transactions where required.
- Keep API contracts documented and testable.

4. BACKEND API IMPLEMENTATION

4.1 API foundation

Create a versioned API under:

/api/v1

Use Laravel's API routing conventions.

Before implementation, map existing web functionality to API endpoints.

Proposed endpoint groups include:

/api/v1/auth

/api/v1/me

/api/v1/dashboard

/api/v1/activities

/api/v1/handovers

/api/v1/incidents

/api/v1/messages

/api/v1/channels

/api/v1/war-rooms

/api/v1/notifications

/api/v1/reports

/api/v1/team

/api/v1/health

These are proposed boundaries only. Adapt them to the actual repository.

4.2 API response standard

Create a consistent response format.

Successful response example:

```
{
  "success": true,
  "data": {},
  "meta": {},
  "message": "Request completed successfully"
}
```

Error response example:

```
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "field": [
      "The field is required."
    ]
  }
}
```

For paginated responses, include pagination metadata.

Follow HTTP semantics:

200: Successful request.

201: Resource created.

204: Successful request with no content.

400: Malformed request where applicable.

401: Unauthenticated.

403: Unauthorized.

404: Resource not found.

409: Conflict.

422: Validation failure.

429: Rate limit exceeded.

500: Unexpected server error.

Never expose stack traces, SQL queries, secrets, or internal exception details in production responses.

4.3 API Resources and validation

Implement or extend:

API Resource classes.

Form Request classes.

API exception handling.

Consistent validation responses.

Pagination.

Filtering and sorting where needed.

Resource relationships.

Date/time serialization using a documented convention, preferably ISO 8601 with explicit timezone handling.

Ensure that sensitive model fields such as passwords, tokens, internal secrets, and private metadata are never serialized.

5. SANCTUM AUTHENTICATION

Implement secure Laravel Sanctum authentication for the mobile application.

5.1 Authentication requirements

Support:

Login with the existing supported credentials.

Logout from the current mobile session.

Fetch authenticated user profile.

Session/token revocation.

Password reset if the existing platform supports it.

Account status checks.

Role and permission retrieval.

Optional device/session management.

Token expiration and revocation strategy.

Use the authentication model and guards already established in the repository.

For native mobile clients, implement a documented token-based authentication flow using Sanctum personal access tokens or another verified Sanctum-compatible approach.

Do not expose tokens in logs or analytics.

5.2 Token security

Store only the necessary token material.

Store mobile tokens using secure platform storage.

Never use SharedPreferences or unencrypted local storage for bearer tokens.

Hash or otherwise protect token-related data in accordance with the chosen Sanctum implementation.

Use HTTPS in production.

Revoke tokens on logout.

Provide a mechanism for administrators to revoke compromised sessions where appropriate.

Do not hardcode credentials or API secrets in the Flutter application.

Configure token abilities only if they can be enforced consistently.

Use server-side authorization regardless of token abilities.

5.3 Authentication endpoints

Implement only endpoints justified by the existing application:

POST /api/v1/auth/login

POST /api/v1/auth/logout

GET /api/v1/me

POST /api/v1/auth/revoke-sessions

Add password reset, refresh, and device management endpoints only when compatible with the actual authentication design.

5.4 Authorization

Preserve existing roles and permissions.

Inspect the application to determine whether it uses:

Laravel Gates.

Policies.

Spatie Laravel Permission.

Custom role management.

Other authorization mechanisms.

Apply the existing authorization rules to API controllers and domain Actions.

Never trust role values, user IDs, or permissions supplied by the mobile client.

Test horizontal and vertical privilege escalation scenarios.

6. IMPLEMENT ALL EXISTING FEATURE MODULES

The mobile app must cover the existing platform's real capabilities, subject to mobile usability and permission constraints.

Do not omit existing core workflows.

MODULE A — Dashboard and operational overview

Build a mobile dashboard that displays the information available to the authenticated user.

Potential capabilities:

Today's shift overview.

Assigned tasks.

Pending activities.

Completed activities.

Incident summary.

Handover status.

Unread messages.

Relevant operational metrics.

Important alerts.

Service health indicators, if implemented in the backend.

Requirements:

Respect user permissions.

Provide loading, empty, error, and offline states.

Use pagination or summarized data where appropriate.

Avoid expensive dashboard queries.

Provide pull-to-refresh.

Avoid exposing confidential information in notifications or screenshots.

MODULE B — Daily shift activity board

Implement mobile workflows for the existing activity board.

Potential operations:

View assigned activities.

View activities by shift/date/status.

Create activities if authorized.

Update activities if authorized.

Assign or delegate activities if authorized.

Mark activities as complete.

Add comments or notes.

View activity history.

Search and filter activities.

Verify the actual activity lifecycle in the repository before implementation.

Prevent unauthorized modification of another user's records.

Use server-side validation and authorization.

MODULE C — Shift handover management

Implement the existing two-way handover workflow.

Potential features:

View current and previous handovers.

View outgoing handover details.

Create or initiate handovers if permitted.

Review incoming handovers.

Sign off on handovers.

Confirm receipt.

View pending sign-offs.

Track handover status.

View audit history.

Handle conflicting or already-signed handovers safely.

Critical requirements:

Preserve the existing handover state machine.

Do not bypass required sign-offs.

Make sign-off requests idempotent where practical.

Protect handover records from unauthorized edits.

Preserve auditability.

MODULE D — Incident management and escalation

Implement mobile incident workflows supported by the backend.

Potential features:

View incidents.

Create incidents if permitted.

Update incident status.

Record severity and priority.

Assign responders.

Add incident notes.

Escalate incidents.

View escalation history.

View incident timelines.

Acknowledge incidents.

Resolve or close incidents according to existing rules.

Participate in war rooms when applicable.

Requirements:

Preserve existing severity and status definitions.

- Ensure that only authorized users can change incident state.
- Use transactions for important state transitions.
- Prevent duplicate submissions.
- Preserve the incident audit trail.
- Do not claim real-time incident updates unless the backend supports them.

MODULE E — Messaging and team communication

Implement existing messaging capabilities.

Potential features:

- Direct messages.
- Team channels.
- War rooms.
- Conversation lists.
- Message history.
- Sending messages.
- Unread counts.
- Mentions, if supported.
- Attachments, if supported.
- Message timestamps.
- Message delivery state, if supported.
- Pagination and lazy loading.

Requirements:

- Inspect the actual messaging models and workflows.
- Do not expose conversations to unauthorized users.
- Validate message content server-side.
- Restrict attachment types and sizes.
- Scan or safely process uploaded files where applicable.
- Apply rate limiting.
- Use secure WebSockets or polling fallback where appropriate.
- Do not build a custom encryption protocol.
- If end-to-end encryption is required, document and design it separately.

MODULE F — Notifications

Implement notification support for the mobile client.

Potential features:

- In-app notification inbox.
- Unread notification count.
- Mark as read.
- Mark all as read where supported.
- Deep links to incidents, activities, or handovers.
- Push notifications.
- Use Firebase Cloud Messaging only after verifying the backend's notification architecture.

Requirements:

Do not put sensitive operational information in push payloads unless explicitly approved.
Store device push tokens securely.
Support token rotation.
Associate tokens with authenticated users.
Revoke or deactivate tokens on logout where appropriate.
Avoid duplicate notifications.
Handle notification permission denial gracefully.

MODULE G — Reports and exports

Implement the existing reporting capabilities in a mobile-friendly format.

Potential features:

View reports.
Filter by date, team, shift, and status where supported.
Display summary metrics.
View charts.
Download authorized reports.
Share or save reports using platform mechanisms where appropriate.

Requirements:

Reuse existing reporting logic.
Enforce access controls on report generation and downloads.
Avoid loading huge datasets directly into mobile memory.
Use asynchronous jobs for expensive reports.
Implement secure, expiring download links where appropriate.
Avoid exposing unprotected export URLs.

MODULE H — Team management

If the existing application supports these operations, implement:

Team directory.
User profiles.
Team membership.
Role visibility.
Availability/status.
Team assignments.
Administrative management features only for authorized roles.
Do not expose confidential employee data.
Do not assume that every web administrative feature belongs in the mobile app.

MODULE I — Health diagnostics and operational system status

Inspect the repository's health endpoints and diagnostic features.

If the platform exposes system health information, implement a restricted mobile health view that may include:

Service availability.
Database health.
Queue status.

Application health.

Relevant system metrics.

Last updated time.

Requirements:

Never expose sensitive infrastructure credentials or internal secrets.

Enforce administrator or appropriate operational permissions.

Avoid exposing unrestricted internal diagnostic endpoints.

Use safe, aggregated health information.

MODULE J — Audit logs

If the application has audit-log functionality, provide an appropriate read-only mobile view for authorized users.

Support:

Filtered audit history.

Actor information according to permissions.

Event type.

Timestamp.

Related resource.

Audit details where safe.

Preserve append-only behavior if the existing audit design requires it.

7. FLUTTER MOBILE APPLICATION

7.1 Technology requirements

Use:

Flutter stable channel.

Dart.

Material 3, adapted to the existing Npontu SRE design.

Riverpod for state management, unless repository or team constraints justify another architecture.

Dio for HTTP networking.

go_router for navigation.

flutter_secure_storage for secure token storage.

Firebase Cloud Messaging if push notifications are implemented.

A charting library only where reports require charts.

A WebSocket client compatible with the Laravel real-time implementation.

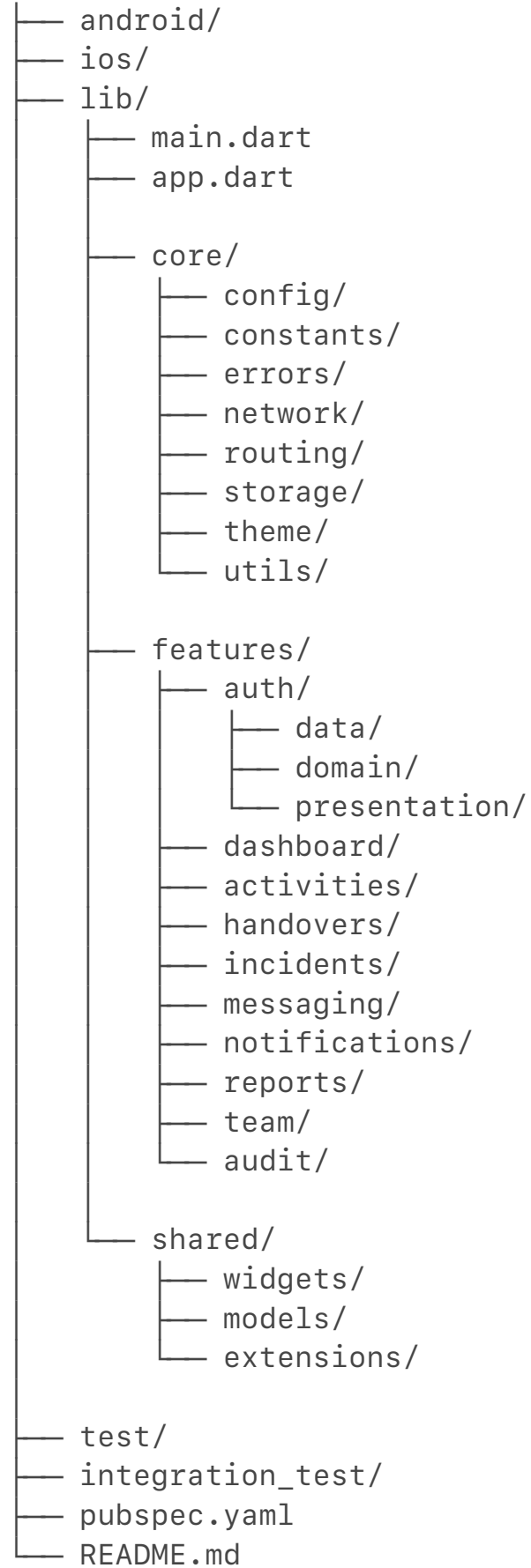
Pin compatible dependency versions and avoid unnecessary packages.

7.2 Flutter architecture

Use a feature-first architecture.

Recommended structure:

npontu_sre_mobile/



Adapt the structure if a better design is justified.

7.3 Mobile UX requirements

Create a professional SRE operations interface.

Requirements:

Responsive layouts.

Material 3 design.

Dark mode and light mode.

Clear status indicators.

Accessible contrast.

Large, touch-friendly controls.

Pull-to-refresh.

Skeleton loading states.

Empty states.

Error recovery.

Offline and reconnecting states.

Confirmation dialogs for destructive actions.

Form validation.

Search and filters.

Deep links from notifications.

Tablet-friendly layouts where practical.

No excessive animations in operational workflows.

Do not display fake metrics or fake monitoring results.

7.4 Navigation

Implement authenticated and unauthenticated navigation.

Potential authenticated navigation:

Home.

Activities.

Handovers.

Incidents.

Messages.

Notifications.

Profile/Settings.

Adapt navigation to actual feature availability and user permissions.

Ensure users cannot access restricted screens merely by manipulating routes.

7.5 Networking layer

Implement:

Centralized Dio client.

Base URL configuration by environment.

Authentication headers.

Request timeouts.

Safe retry policies.

Error mapping.
Connectivity handling.
Pagination support.
Request cancellation where useful.
Consistent serialization and deserialization.
Logging that redacts credentials and sensitive information.
Do not retry non-idempotent operations blindly.

8. API CONTRACT AND DOCUMENTATION

Create and maintain API documentation.

Preferred approach:

OpenAPI 3.x specification.

Swagger UI or an equivalent documentation viewer.

Document authentication.

Document request and response schemas.

Document validation errors.

Document pagination.

Document authorization requirements.

Document status transitions.

Document rate limits.

Document versioning and deprecation policy.

Suggested location:

docs/api/openapi.yaml

If an API documentation tool is already present, integrate with it instead of introducing unnecessary tooling.

Add contract tests for critical endpoints.

Ensure the Flutter models reflect the documented API schema.

9. SECURITY AND DEVSECOPS

Treat security as a first-class requirement.

9.1 Backend security

Implement or verify:

- HTTPS in production.
- Strict authorization on all protected API routes.
- CSRF protection for stateful web routes.
- Rate limiting on login and sensitive endpoints.
- Input validation.
- Output sanitization where applicable.
- SQL injection protection through safe query practices.
- Secure file uploads.
- MIME type and size validation.
- Secure download authorization.
- Secure password handling.
- No secrets in source control.
- Safe error responses.
- Security headers where applicable.
- CORS configuration restricted to legitimate clients.
- Session/token revocation.
- Audit logging of security-sensitive operations.

9.2 Mobile security

- Secure token storage.
- No secrets embedded in the application.
- No hardcoded production credentials.
- HTTPS-only production communication.
- Safe handling of deep links.
- No sensitive data in logs.
- No sensitive data in crash reports.
- Secure handling of screenshots where required by organizational policy.
- Appropriate certificate validation.
- Avoid insecure SSL bypasses.
- Consider platform protections against screenshots for especially sensitive screens, subject to product requirements.
- Do not implement certificate pinning unless its operational implications and update strategy are understood.

9.3 Supply-chain security

- Audit Composer dependencies.
- Audit Flutter dependencies.
- Audit npm dependencies where applicable.
- Keep lockfiles under version control.
- Scan for secrets.
- Scan for vulnerable dependencies.
- Use Dependabot or an equivalent tool if appropriate.
- Pin production build dependencies where practical.

9.4 Threat model

Create:

docs/security/mobile-threat-model.md

Cover:

Stolen mobile token.

Unauthorized API access.

Broken object-level authorization.

Account takeover.

Insecure file upload.

WebSocket authorization failures.

Data leakage through push notifications.

Malicious deep links.

Replay of mutating requests.

Excessive API requests.

Lost or compromised devices.

Sensitive information exposure in logs.

Document mitigations and residual risks.

10. DATABASE AND BACKEND PERFORMANCE

Do not unnecessarily modify existing database schemas.

Before adding migrations:

Inspect the current schema.

Confirm whether the required data already exists.

Identify missing indexes.

Assess relationship loading and N+1 queries.

Check existing constraints.

Document migration impact.

Requirements:

Add indexes where justified.

Use eager loading appropriately.

Paginate large collections.

Avoid loading unnecessary columns.

Use queues for expensive operations.

Use Redis only where its operational value is justified.

Preserve existing data.

Create reversible migrations where practical.

Test migrations on a non-production database.

Do not introduce a new database engine merely to support Flutter.

11. OFFLINE SUPPORT AND NETWORK RESILIENCE

SRE staff may work with unreliable mobile connectivity.

Implement a realistic network strategy.

Minimum requirements:

Display connection state.

Handle timeouts gracefully.

Cache appropriate read-only data.

Allow safe refresh.

Preserve unsent drafts where appropriate.

Prevent duplicate submissions.

Clearly indicate stale data.

Handle server-side conflicts.

Provide retry actions.

Avoid claiming full offline functionality unless it is implemented and tested.

If offline activity creation or editing is required, design an explicit synchronization strategy with:

Local operation identifiers.

Idempotency keys.

Conflict handling.

Sync status.

Server reconciliation.

Data retention rules.

Do not implement a complex offline-first system without verifying the actual requirements.

12. REAL-TIME FEATURES

Inspect whether the existing project uses:

Laravel broadcasting.

Reverb.

Pusher.

WebSockets.

Server-sent events.

Polling.

If real-time updates are already supported:

Implement the Flutter client integration.

Authenticate private channels securely.

Subscribe only to authorized channels.

Handle reconnects.

Handle duplicate events.

Update local state safely.

Provide polling fallback where appropriate.

If real-time support does not exist, implement a minimal, maintainable solution only where it provides clear value.

Do not add WebSockets merely for appearance.

13. TESTING STRATEGY

Testing is mandatory.

13.1 Backend tests

Create or extend Laravel tests for:

Login and logout.

Authenticated profile retrieval.

Token revocation.

API validation.

Authorization.

Role-based access.

Activity CRUD and lifecycle.

Shift handovers.

Two-way sign-offs.

Incident state transitions.

Escalation rules.

Messaging access controls.

Notification registration.

Report authorization.

Audit-log integrity.

File upload restrictions.

Rate limiting.

Idempotency behavior.

Pagination.

API error formats.

Use PHPUnit or Pest according to the existing project.

13.2 Flutter tests

Implement:

Unit tests

API models.

JSON serialization.

Repositories.

Validators.

State notifiers/providers.

Error mapping.

Authentication state transitions.

Widget tests

Login.

Dashboard.

Activity list.

Activity form.

Handover sign-off.

Incident details.

Messaging interface.

Notification inbox.

Error and empty states.

Integration tests

Test critical workflows against a test backend or controlled integration environment:

Login.

Fetch dashboard.

View activities.

Update an authorized activity.

Complete a handover sign-off.

View and update an incident.

Send a message if supported.

Receive a notification.

Logout and verify protected access is revoked.

13.3 Security testing

Test:

Unauthorized resource access.

Cross-user data access.

Invalid tokens.

Revoked tokens.

Expired sessions.

Invalid payloads.

Excessive requests.

File upload attacks.
Sensitive data leakage.
API route exposure.
WebSocket channel authorization.

13.4 Test quality

Do not simply write tests that assert that the application returns HTTP 200.
Test business rules, permissions, state transitions, error cases, and data integrity.
Run all existing tests before and after major changes.
Do not weaken or delete existing tests to make the build pass.

14. CI/CD PIPELINE

Set up or improve automated CI/CD.

Backend pipeline

On every pull request:
Install PHP dependencies.
Run code style checks.
Run static analysis if configured.
Run Laravel/PHP tests.
Run security/dependency checks.
Validate migrations and configuration where practical.

Flutter pipeline

On every pull request:
Run flutter pub get.
Run dart format checks.
Run flutter analyze.
Run unit tests.
Run widget tests.
Build a debug or release-compatible artifact where feasible.

Build environments

Use environment-specific configuration:
Development.
Staging.
Production.
Never commit:
API secrets.

Signing keys.
Firebase private credentials.
Production .env files.
Keystores.
iOS certificates.
Access tokens.
Document secret management and CI variables.

15. DEPLOYMENT

15.1 Backend deployment

Inspect the existing deployment strategy before changing it.

Prepare a production deployment guide covering:

PHP version.
Laravel configuration.
APP_KEY management.
Database configuration.
Cache configuration.
Queue workers.
Scheduler.
Storage permissions.
File storage.
HTTPS.
API URL.
CORS.
Rate limiting.
Monitoring.
Error reporting.
Database backups.
Rollback procedures.
Migration procedures.
Health checks.

If the current hosting platform is unsuitable for the required workload, document alternatives and trade-offs rather than silently migrating infrastructure.

15.2 Flutter Android deployment

Prepare:

Android application ID.
App display name.

App icon.
Splash screen.
Release build configuration.
Signing configuration.
Secure keystore handling.
Environment configuration.
Google Play App Bundle build.
Internal testing track deployment instructions.
Privacy policy requirements.
Permissions review.
Never commit the Android signing keystore.

15.3 Flutter iOS deployment

Prepare:
Bundle identifier.
App icon.
Launch configuration.
Signing and provisioning requirements.
Apple Developer configuration.
Release build instructions.
TestFlight deployment instructions.
App Store privacy declarations.
Required permissions descriptions.
iOS release builds require macOS/Xcode or an appropriate macOS CI environment. Do not claim an iOS release has been built unless it has actually been built.

15.4 Environment configuration

The mobile app must support configurable API endpoints.

Example:

Development:

<https://staging-api.example.com>

Production:

<https://api.example.com>

Use the actual deployed API domains only after they are verified.

Do not hardcode placeholder URLs into a production release.

16. OBSERVABILITY AND OPERATIONS

Extend the platform's operational visibility to the new mobile client.

Implement where appropriate:

Structured backend logs.

API request correlation IDs.

Error tracking.

Performance monitoring.

Queue monitoring.

Database health monitoring.

Mobile crash reporting.

Mobile performance monitoring.

API latency tracking.

Authentication failure monitoring.

Security event alerts.

Avoid logging:

Access tokens.

Passwords.

Secrets.

Sensitive message contents.

Confidential employee information.

Create:

docs/observability.md

Document dashboards, alerts, and incident response procedures.

17. UI/UX DESIGN SYSTEM

Create a consistent Npontu SRE visual identity.

First inspect the existing Blade/Livewire UI and reuse:

Color palette.

Typography.

Spacing.

Status colors.

Button styles.

Card styles.

Navigation patterns.

Brand assets.

Accessibility conventions.

Build Flutter equivalents rather than copying web layouts literally.

The mobile UI should feel like a professional operations product.

Prioritize:

Clarity.

Speed.

Readability.

Operational context.

Low cognitive load.

Accessible controls.

Clear status and severity indicators.

Do not fabricate logos, brand assets, or visual metrics.

18. DEVELOPMENT WORKFLOW

Implement in incremental, reviewable phases.

Phase 0 — Audit

Inspect repository.

Produce architecture and security audit.

Identify API gaps.

Identify dependencies.

Document risks.

Deliverable:

docs/mobile-expansion-audit.md

Phase 1 — API foundation

Configure API versioning.

Implement authentication.

Implement response conventions.

Implement API Resources and Form Requests.

Add initial API tests.

Create API documentation.

Deliverables:

Laravel API foundation.

Sanctum authentication.

API tests.

OpenAPI specification.

Phase 2 — Core mobile features

Initialize Flutter project.

Configure environments.

Implement authentication.

Implement navigation.
Implement dashboard.
Implement activities.
Implement handovers.
Implement secure local storage.

Deliverables:

Working authenticated Flutter app.
Core feature workflows.
Automated tests.

Phase 3 — Operations and communications

Incidents.
Escalation workflows.
Messaging.
War rooms.
Notifications.
Real-time updates if justified.

Deliverables:

Operationally useful mobile application.
Messaging and notification integration.
Security tests.

Phase 4 — Reports and administration

Reports.
Exports.
Team management.
Audit views.
Health diagnostics, subject to permissions.

Deliverables:

Feature-complete mobile workflows according to the audit.
API documentation updates.

Phase 5 — Hardening and deployment

Security testing.
Performance testing.
Offline and resilience testing.
CI/CD.
Staging deployment.
Android internal testing.
iOS TestFlight preparation.
Production readiness review.

Deliverables:

Release candidates.
Deployment documentation.

Test reports.
Security review.
Rollback plan.

19. DEFINITION OF DONE

The project is complete only when all applicable criteria are met.

Backend

- ☐ API endpoints are versioned.
- ☐ Existing domain logic is reused where appropriate.
- ☐ Authentication is secure.
- ☐ Authorization is enforced.
- ☐ Validation is implemented.
- ☐ API responses are consistent.
- ☐ Sensitive data is protected.
- ☐ Critical workflows have automated tests.
- ☐ API documentation is available.
- ☐ Existing web functionality still works.
- ☐ Database migrations are reviewed and tested.

Flutter

- ☐ Android and iOS project configuration is present.
- ☐ Authentication works against the real backend.
- ☐ Core existing workflows are supported.
- ☐ Loading, error, empty, and offline states are handled.
- ☐ Token storage is secure.
- ☐ Navigation respects authentication and permissions.
- ☐ API errors are handled correctly.
- ☐ Unit and widget tests are present.
- ☐ Critical integration tests are present.
- ☐ Accessibility and responsive layouts are reviewed.

Security

- ☐ Authorization tests pass.
- ☐ No secrets are committed.
- ☐ Dependencies are audited.
- ☐ File uploads are restricted.
- ☐ Sensitive logs are removed.
- ☐ Push notification privacy is reviewed.

☐ Threat model is documented.

Deployment

- ☐ Staging deployment is documented or verified.
- ☐ Backend production configuration is documented.
- ☐ Android release build is verified where possible.
- ☐ iOS build requirements are documented.
- ☐ Signing credentials are protected.
- ☐ Monitoring and rollback procedures are documented.

Do not mark unchecked items as complete.

20. REQUIRED FINAL DELIVERABLES

At the end of the implementation, provide a comprehensive report containing:

Executive summary.

Existing architecture findings.

Changes made to the Laravel backend.

New API endpoints.

Authentication implementation details.

Database changes.

Flutter application architecture.

Mobile features implemented.

Features deferred and why.

Security improvements.

Tests executed and exact results.

Known issues and limitations.

Environment variables required.

Deployment instructions.

Android build instructions.

iOS build instructions.

Recommended next steps.

Create or update:

`docs/mobile-expansion-audit.md`

`docs/mobile-api.md`

`docs/security/mobile-threat-model.md`

`docs/deployment/mobile-deployment.md`

`docs/mobile-development.md`

Also update the main README with the new architecture and setup instructions.

21. AGENT EXECUTION RULES

Work in small, logical commits or clearly separated changes.

Keep the existing web application functional.

Do not silently overwrite user changes.

Do not make destructive database changes.

Do not remove dependencies unless their removal is justified.

Do not expose secrets in output.

Do not use mock data as a substitute for real backend integration.

Do not mark features as complete without testing.

If a build or test fails, investigate the root cause.

If a feature is blocked by missing infrastructure, document the blocker and continue with independent work.

Prefer maintainable, production-quality code over quick hacks.

Review all security-sensitive code carefully.

Before finalizing, run the relevant tests, static analysis, and build checks.

BEGIN EXECUTION

Start by auditing the repository at:

<https://github.com/mhiskall282/npontu-technologies-sre>

Do not start by creating a generic Flutter demo.

First, understand the existing Laravel architecture and produce the audit report. Then implement the mobile expansion incrementally, reusing existing business logic and preserving the web application.

Your final response must clearly distinguish:

What you inspected.

What you implemented.

What you tested.

What remains incomplete.

What requires manual configuration or human approval.

Proceed with engineering discipline and production-readiness as the primary goals.